

Naro Fashion — Production Deployment Guide

Complete step-by-step guide with troubleshooting

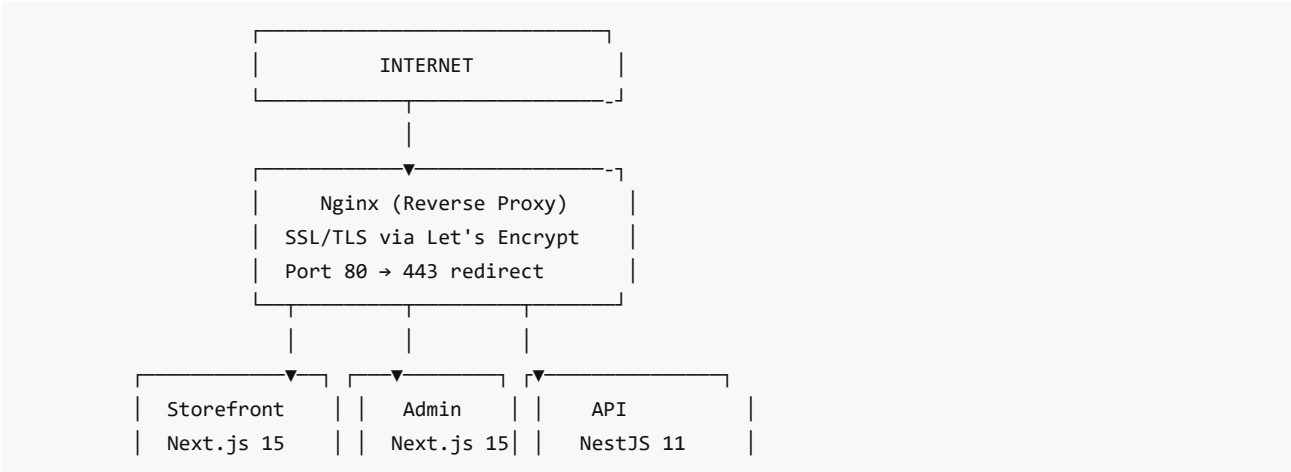
Version	2.0
Date	April 2026
Author	Rogath J. Urío
Platform	Multi-Tenant SaaS E-Commerce
Stack	Next.js 15 + NestJS 11 + PostgreSQL + Prisma

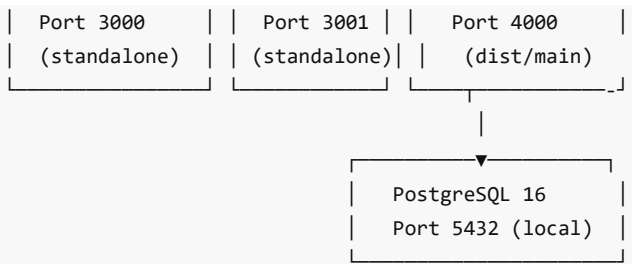
This guide is based on real deployment experience. Every command has been run, every error has been encountered and solved. A new administrator with basic Linux knowledge should be able to deploy the entire platform from scratch using only this document.

Table of Contents

- [Prerequisites](#)
- [Part 1: VPS Provisioning](#)
- [Part 2: Server Setup](#)
- [Part 3: Application Deployment](#)
- [Part 4: Build Troubleshooting](#)
- [Part 5: PM2 Configuration](#)
- [Part 6: Nginx Configuration](#)
- [Part 7: DNS Setup](#)
- [Part 8: SSL Certificates](#)
- [Part 9: Email Setup \(Brevo\)](#)
- [Part 10: Tenant Configuration](#)
- [Part 11: CI/CD Pipeline](#)
- [Part 12: Firewall and Security](#)
- [Part 13: Post-Deployment Verification](#)
- [Part 14: Known Issues and Solutions](#)
- [Part 15: Updating the Application](#)
- [Part 16: Cost Summary](#)
- [Appendix A: All Environment Variables Reference](#)
- [Appendix B: Quick Reference Commands](#)

Architecture Overview





Production URLs:

Domain	Service	Internal Port
narofashion.co.tz	Storefront (customer shop)	3000
admin.narofashion.co.tz	Admin dashboard	3001
api.narofashion.co.tz	REST API (/api/v1)	4000

Prerequisites

Before you begin, you need:

Requirement	Details
Domain name	A registered domain (e.g., narofashion.co.tz). For .co.tz domains, use Habari Node (habari.co.tz) or any tzNIC-accredited registrar
VPS account	Vultr, Hetzner, DigitalOcean, or Linode account with a credit/debit card
GitHub access	Account with access to github.com/rjuriio/naro-fashion (or your fork)
SSH client	Git Bash (Windows) or Terminal (Mac/Linux). Do NOT use PowerShell for SSH — it has password paste issues
SSH key pair	Ed25519 key pair (generated in the steps below if you do not have one)
Email service	Brevo free account (for transactional emails — 300/day free tier)
Time estimate	First deployment: 2-3 hours. Subsequent deployments: 5-10 minutes

Part 1: VPS Provisioning

1.1 Generate SSH Key (Windows)

IMPORTANT: Use Git Bash, not PowerShell. PowerShell cannot paste passwords into SSH prompts, which will frustrate you during server setup.

Open **Git Bash** and run:

```
# Generate an Ed25519 SSH key
ssh-keygen -t ed25519 -C "your-email@example.com"
# Press Enter to accept default location (~/.ssh/id_ed25519)
# Enter a passphrase (or leave empty for no passphrase)
```

```
# Display your public key (copy this for the VPS)
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output (starts with `ssh-ed25519 AAAA...`). You will paste this into your VPS provider.

1.2 Create VPS Server

Option A: Vultr (Recommended — \$12/mo)

1. Go to <https://www.vultr.com> and create an account
2. Navigate to **Products > Compute** > click **Deploy +**
3. Select **Cloud Compute > Shared CPU**
4. Configure:

Setting	Value
Location	Frankfurt, DE (closest to East Africa with good latency)
Image	Ubuntu 24.04 LTS x64
Plan	High Frequency: vhf-1c-2gb (1 vCPU, 2GB RAM, 64GB NVMe) — \$12/mo
SSH Key	Click "Add New" and paste your public key from step 1.1
Hostname	naro-fashion-prod

5. Click **Deploy Now**
6. Wait 1-2 minutes for the server to provision
7. Note the **IP address** from the server details page (e.g., `80.240.30.107`)

Why Frankfurt? It is the closest major data center region to East Africa. Latency from Dar es Salaam is typically 120-180ms, compared to 200-300ms for US-based servers.

Option B: Hetzner (\$7.50/mo — cheapest)

1. Go to <https://console.hetzner.com>
2. **Note:** Hetzner requires ID verification (passport upload). This can take 1-24 hours.
3. Create Project > Add Server > Ubuntu 24.04, CX22 (2 vCPU, 4GB RAM), Falkenstein
4. Add your SSH key during creation

Option C: DigitalOcean (\$12/mo)

1. Go to <https://www.digitalocean.com>
2. Create Droplet > Ubuntu 24.04, Basic, Regular \$12/mo, Frankfurt
3. Add your SSH key during creation

1.3 First SSH Connection

Use Git Bash (not PowerShell):

```
ssh root@YOUR_VPS_IP
# Example: ssh root@80.240.30.107

# If you get a fingerprint prompt, type "yes"
# If using password auth (no SSH key), you'll need the root password from VPS dashboard
```

If you used a password during VPS creation and cannot paste it:

- In Git Bash, right-click to paste (Ctrl+V does not work in Git Bash)
- Some terminals: use Shift+Insert to paste

1.4 Add SSH Key to Server (if not added during provisioning)

If you created the VPS without an SSH key and are using password authentication:

```
# On the server, paste your public key:
mkdir -p ~/.ssh
echo "ssh-ed25519 AAAA... your-email@example.com" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

From this point forward, you can SSH without a password.

Part 2: Server Setup

All commands in this section run **on the VPS as root** (connected via SSH).

2.1 System Update and Essential Packages

```
# Update package lists and upgrade all installed packages
apt update && apt upgrade -y

# Install essential packages
apt install -y \
  curl \
  git \
  build-essential \
  nginx \
  certbot \
  python3-certbot-nginx \
  ufw \
  htop \
  unzip
```

What each package does:

Package	Purpose
curl	HTTP client for downloading installers
git	Clone the repository
build-essential	C/C++ compiler for native Node.js modules
nginx	Reverse proxy / web server
certbot + python3-certbot-nginx	Free SSL certificates from Let's Encrypt
ufw	Uncomplicated Firewall
htop	Process/memory monitor
unzip	Archive extraction

2.2 Install Node.js 22 LTS

```
# Add NodeSource repository for Node.js 22
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -

# Install Node.js
apt install -y nodejs

# Verify
node -v
# Expected output: v22.x.x
```

2.3 Install pnpm and PM2

```
# Install pnpm (package manager – the project uses pnpm workspaces)
npm install -g pnpm@10

# Install PM2 (process manager – keeps apps running and restarts on crash)
npm install -g pm2

# Verify
pnpm -v    # Expected: 10.x.x
pm2 -v     # Expected: 5.x.x or 6.x.x
```

2.4 Install PostgreSQL 16

```
# Install PostgreSQL
apt install -y postgresql postgresql-contrib

# Enable and start the service
systemctl enable postgresql
systemctl start postgresql

# Verify
psql --version
# Expected: psql (PostgreSQL) 16.x
```

2.5 Create Database and User

```
# Generate a strong password (save this – you will need it for .env)
openssl rand -base64 32
# Example output: a1B2c3D4e5F6g7H8i9J0k1L2m3N4o5P6
# SAVE THIS PASSWORD!

# Enter PostgreSQL as the postgres superuser
sudo -u postgres psql
```

Inside the PostgreSQL shell:

```
-- Replace <YOUR_STRONG_PASSWORD> with the password you generated above
CREATE USER naro_admin WITH PASSWORD '<YOUR_STRONG_PASSWORD>';
CREATE DATABASE naro_fashion OWNER naro_admin;
GRANT ALL PRIVILEGES ON DATABASE naro_fashion TO naro_admin;
```

```
-- Verify
\l
-- You should see naro_fashion in the list, owned by naro_admin

-- Exit
\q
```

2.6 Generate JWT Secrets

Generate two separate secrets — one for access tokens and one for refresh tokens:

```
# JWT_SECRET (for access tokens)
openssl rand -base64 64 | tr -dc 'a-zA-Z0-9' | head -c 64
# SAVE THIS! Example: aB3dE5fG7hI9jK1lM3nO5pQ7rS9tU1vW3xY5zA7bC9dE1fG3hI5jK7

# JWT_REFRESH_SECRET (for refresh tokens – must be different from above)
openssl rand -base64 64 | tr -dc 'a-zA-Z0-9' | head -c 64
# SAVE THIS! Example: zY9xw7vU5tS3rQ1p09nM7lK5jI3hG1fE9dC7bA5zY3xW1vU9tS7rQ5
```

2.7 Add Swap Space (Critical for 2GB RAM)

The build process uses significant memory. Without swap, it will fail on a 2GB server.

```
# Create a 2GB swap file
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile

# Make it permanent (survives reboot)
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab

# Verify
free -h
# You should see ~2.0G under Swap
```

Part 3: Application Deployment

3.1 Clone the Repository

```
# Create the application directory
mkdir -p /var/www/naro-fashion

# Clone the repo
cd /var/www/naro-fashion
git clone https://github.com/rjurio/naro-fashion.git .
# Note the dot (.) at the end – clones into current directory, not a subdirectory
```

If the repo is private, you need to authenticate:

```
# Option A: HTTPS with personal access token
git clone https://<YOUR_GITHUB_TOKEN>@github.com/rjurio/naro-fashion.git .
```

```
# Option B: SSH (add server's SSH key to GitHub)
cat ~/.ssh/id_ed25519.pub
# Copy this and add it to GitHub > Settings > SSH keys
git clone git@github.com:rjurio/naro-fashion.git .
```

3.2 Create Environment Files

You need 4 environment files. Create them carefully — missing or wrong values are the most common cause of deployment failures.

Root .env file

```
cat > /var/www/naro-fashion/.env << 'EOF'
NODE_ENV=production

DATABASE_URL="postgresql://naro_admin:<DB_PASSWORD>@localhost:5432/naro_fashion?schema=public"

JWT_SECRET="<YOUR_JWT_SECRET>"
JWT_REFRESH_SECRET="<YOUR_JWT_REFRESH_SECRET>"
JWT_EXPIRATION="15m"
JWT_REFRESH_EXPIRATION="7d"

STOREFRONT_URL="https://yourdomain.com"
ADMIN_URL="https://admin.yourdomain.com"
API_URL="https://api.yourdomain.com"
NEXT_PUBLIC_API_URL="https://api.yourdomain.com/api/v1"

SMTP_HOST=""
SMTP_PORT="587"
SMTP_USER=""
SMTP_PASS=""
SMTP_FROM="noreply@yourdomain.com"

INSTAGRAM_BUSINESS_ACCOUNT_ID=""
FACEBOOK_APP_ID=""
EOF
```

Now edit the file and replace all placeholder values:

```
nano /var/www/naro-fashion/.env
```

Replace:

- `<DB_PASSWORD>` — the PostgreSQL password from Part 2.5
- `<YOUR_JWT_SECRET>` — the first secret from Part 2.6
- `<YOUR_JWT_REFRESH_SECRET>` — the second secret from Part 2.6
- `yourdomain.com` — your actual domain (e.g., `narofashion.co.tz`)
- SMTP fields — leave empty for now, fill in after Part 9 (Brevo setup)

Save and exit (`Ctrl+X` , then `Y` , then `Enter`).

Copy .env to sub-packages that need it

```
# The API and database packages need the same DATABASE_URL and JWT secrets
cp /var/www/naro-fashion/.env /var/www/naro-fashion/packages/database/.env
```

```
cp /var/www/naro-fashion/.env /var/www/naro-fashion/apps/api/.env
```

Storefront .env.local

```
cat > /var/www/naro-fashion/apps/storefront/.env.local << 'EOF'
NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api/v1
NEXT_PUBLIC_TENANT_SLUG=naro-fashion
EOF
```

Replace `yourdomain.com` with your actual domain.

Why `NEXT_PUBLIC_TENANT_SLUG` ? During local development (no custom domain), the storefront resolves the tenant by slug instead of domain. On production, the storefront resolves by domain (set in Part 10), but the slug is kept as a fallback.

Admin .env.local

```
cat > /var/www/naro-fashion/apps/admin/.env.local << 'EOF'
NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api/v1
EOF
```

Replace `yourdomain.com` with your actual domain.

3.3 Install Dependencies

```
cd /var/www/naro-fashion

# Install all monorepo dependencies
pnpm install
```

This takes 2-5 minutes. It installs dependencies for all 6 packages (storefront, admin, api, database, shared, ui).

Install multer explicitly

```
# multer is used for file uploads in the API but pnpm strict hoisting
# does not auto-resolve it. This MUST be installed explicitly or the
# API will crash with "Cannot find module 'multer'" at startup.
pnpm add multer @types/multer --filter api
```

Why is this needed? *pnpm* uses strict dependency isolation. Unlike *npm*, packages can only access dependencies they explicitly declare. The API uses *multer* for file uploads (product images, 3D models, payment icons), but it was originally a transitive dependency. Without explicit installation, the API crashes immediately on startup.

3.4 Set Up the Database

```
cd /var/www/naro-fashion/packages/database

# Generate the Prisma client (creates TypeScript types from schema)
npx prisma generate

# Push the schema to the database (creates all 57+ tables)
npx prisma db push --accept-data-loss
```



```
# Go back to project root
cd /var/www/naro-fashion
```

What does `--accept-data-loss` do? It tells Prisma to proceed even if the schema changes would drop data. On a fresh database this is safe. On subsequent deployments, it handles column renames or type changes that could lose existing data.

3.5 Run Migration and Seed Scripts

```
cd /var/www/naro-fashion

# Create the first tenant, platform admin, subscription plans, and modules
node packages/database/prisma/migrate-to-multi-tenant.js

# Seed tenant-specific data (site settings, CMS pages, payment methods)
node packages/database/prisma/seed-tenant.js
```

What these scripts create:

Script	Creates
migrate-to-multi-tenant.js	Default tenant ("Naro Fashion"), platform admin (platform@naro.co.tz / Admin123), tenant admin (admin@narofashion.co.tz / admin123), 3 subscription plans (Starter/Business/Enterprise), all module definitions
seed-tenant.js	Site settings, default CMS pages (About, Contact, FAQ, Terms, Privacy, Size Guide, Shipping, Returns), payment methods (Visa, Mastercard, M-Pesa, Tigo Pesa, Airtel Money, etc.)

3.6 Build All Applications

This is the most critical step. The build compiles TypeScript for all 3 applications.

```
cd /var/www/naro-fashion

# Build all apps via Turborepo
pnpm build
```

Expected output on success:

```
Tasks:    3 successful, 3 total
Cached:   0 cached, 3 total
Time:     2m 30s (approximate)
```

If the build fails, proceed to Part 4 (Build Troubleshooting). The build process is where most first-time deployment issues surface.

Part 4: Build Troubleshooting

This section documents every build error encountered during real deployments and the exact fix for each one. If your `pnpm build` succeeds on the first try, skip this section.

4.1 "Module not found: @/types/model-viewer"

Error message:

```
Module not found: Can't resolve '@/types/model-viewer'
```

Root cause: The `model-viewer.d.ts` type declaration files must exist in both Next.js apps. These files tell TypeScript about the `<model-viewer>` web component (Google's 3D viewer). If they are missing, the build fails because TypeScript does not know what `<model-viewer>` is.

Fix: Create the type declaration files. They must use `declare module 'react'` (not `declare namespace JSX`) because React 19 changed how JSX type augmentation works.

Create `apps/admin/types/model-viewer.d.ts` :

```
/* eslint-disable @typescript-eslint/no-empty-object-type */
import type React from 'react';

declare module 'react' {
  namespace JSX {
    interface IntrinsicElements {
      'model-viewer': React.DetailedHTMLProps<
        React.HTMLAttributes<HTMLElement> & {
          src?: string;
          alt?: string;
          poster?: string;
          'camera-controls'?: boolean | string;
          'auto-rotate'?: boolean | string;
          ar?: boolean | string;
          'ar-modes'?: string;
          loading?: 'auto' | 'lazy' | 'eager';
          'shadow-intensity'?: string;
        },
        HTMLElement
      >;
    }
  }
}

export {};
```

Create `apps/storefront/types/model-viewer.d.ts` :

```
import "react";

declare module "react" {
  namespace JSX {
    interface IntrinsicElements {
      "model-viewer": React.DetailedHTMLProps<
        React.HTMLAttributes<HTMLElement> & {
          src?: string;
          alt?: string;
          poster?: string;
          "camera-controls"?: boolean | string;
          "auto-rotate"?: boolean | string;
          ar?: boolean | string;
          "ar-modes"?: string;
          loading?: "auto" | "lazy" | "eager";
          style?: React.CSSProperties;
          "shadow-intensity"?: string;
          "environment-image"?: string;
          exposure?: string;
        }
      >;
    }
  }
}
```

```

    },
    HTMLElement
  >;
}
}
}

```

Why declare module 'react' instead of declare namespace JSX ? React 19 moved the JSX namespace inside the React module. If you use the old `declare namespace JSX` pattern, TypeScript ignores your augmentation silently and the build still fails with "Property 'model-viewer' does not exist on type 'JSX.IntrinsicElements'".

4.2 "use client" directive must be placed before other expressions"

Error message:

```
Error: "use client" directive must be placed before other expressions.
Move the directive to the top of the file to resolve this issue.
```

Root cause: In a file that has `"use client"` at the top (a React Client Component), you also added `export const dynamic = "force-dynamic"` — but you placed the `export` before the `"use client"` directive.

Fix: The `"use client"` directive must ALWAYS be the very first line of the file. If you need `export const dynamic`, put it after:

```
// WRONG:
export const dynamic = "force-dynamic";
"use client";

// CORRECT:
"use client";
export const dynamic = "force-dynamic";
```

4.3 "Error occurred prerendering page /auth/login"

Error message:

```
Error occurred prerendering page "/auth/login"
Error: localStorage is not defined
```

or

```
Error occurred prerendering page "/dashboard"
Error: window is not defined
```

Root cause: Next.js tries to prerender (generate static HTML for) all pages at build time. Pages that use `localStorage`, `window`, `document`, or other browser-only APIs crash during this prerender because they run on the server (Node.js) where these APIs do not exist.

Fix: Add `export const dynamic = "force-dynamic"` to the root `app/layout.tsx` of BOTH Next.js apps. This tells Next.js to never prerender any page — all pages are rendered on-demand at request time.

Both layout files should start like this:

```
export const dynamic = "force-dynamic";
import type { Metadata } from "next";
// ... rest of imports
```

This line is already present in the codebase. If it is missing (e.g., after a merge conflict), add it back as the very first line of:

- `apps/storefront/app/layout.tsx`
- `apps/admin/app/layout.tsx`

4.4 "Module not found: Can't resolve 'multer'"

Error message (at runtime, not build time):

```
Error: Cannot find module 'multer'
```

The API starts but immediately crashes. PM2 shows the app restarting repeatedly (restart count climbs into hundreds or thousands).

Root cause: pnpm uses strict dependency hoisting. The `multer` package (used for file uploads) must be explicitly listed in `apps/api/package.json`. Unlike npm, pnpm does not let packages access dependencies they did not explicitly declare.

Fix:

```
cd /var/www/naro-fashion
pnpm add multer @types/multer --filter api
pm2 restart naro-api
```

This adds `multer` to `apps/api/package.json` directly. Once done, it persists across `pnpm install` runs.

4.5 ReactQuill ref prop error

Error message:

```
Warning: Function components cannot be given refs.
Attempts to access this ref will fail.
```

or a build error about `ref` on a dynamically imported component.

Root cause: `react-quill-new` is loaded via `next/dynamic` with `{ ssr: false }`. Dynamic imports wrap the component in a way that does not support React `ref` forwarding.

Fix: Do not use the `ref` prop on the dynamically imported ReactQuill component. If you need to access the editor instance, use a callback or state instead.

4.6 TypeScript "Property X does not exist"

Error message:

```
Type error: Property 'imageUrl' does not exist on type 'Category'.
Type error: Property 'totalSpent' does not exist on type 'User'.
```

Root cause: TypeScript strict mode catches missing interface fields. This happens when the API returns fields that are not declared in the frontend TypeScript interfaces.

Fix (proper): Add the missing field to the TypeScript interface in `packages/shared/` or the local type definition.

Fix (emergency, first deploy only): If you have many TypeScript errors and need to get the site up quickly, temporarily set `ignoreBuildErrors: true` in both Next.js config files:

```
// apps/storefront/next.config.js and apps/admin/next.config.js
typescript: { ignoreBuildErrors: true },
```

IMPORTANT: Fix the actual TypeScript errors and set this back to `false` afterward. Running with `ignoreBuildErrors: true` hides real bugs.

The current codebase has all TypeScript errors fixed and runs with `ignoreBuildErrors: false`.

4.7 Next.js Standalone Mode — Blank Pages

Symptom: The site loads but shows a completely blank/white page, or CSS/JS assets return 404.

Root cause: Next.js `standalone` output mode creates a minimal server at `.next/standalone/apps/<name>/server.js` that does NOT include static assets. You must manually copy them after every build.

Fix: After every `pnpm build`, copy static files:

```
# Storefront
cp -r apps/storefront/.next/static apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public

# Admin
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public
```

This is included in the PM2 setup (Part 5) and the deploy script (Part 11). If you ever build manually, you must run these copy commands.

4.8 Prisma EPERM on OneDrive (Windows Development Only)

Error message (Windows only):

```
EPERM: operation not permitted, unlink '..\..\prisma\client\..'
```

Root cause: OneDrive file sync locks files that Prisma tries to overwrite during `prisma generate`.

Fix: Retry the command — it usually works on the second attempt. Or use `prisma db push` which triggers `generate` internally with better file handling. This is a Windows development issue only — it does not happen on the Linux production server.

4.9 Out of Memory During Build

Symptom: Build process is killed by the OS, or you see `ENOMEM` or `Killed` in the output.

Root cause: 2GB RAM is tight for building 3 applications simultaneously. Without swap, the OOM killer terminates the build process.

Fix: Ensure swap is enabled (Part 2.7):

```
free -h
# If no swap line, create it:
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab
```

If the build still fails with swap, you can increase Node.js memory:

```
NODE_OPTIONS="--max-old-space-size=3072" pnpm build
```

Part 5: PM2 Configuration

PM2 is the process manager that keeps all 3 applications running, restarts them on crash, and starts them on server boot.

5.1 Create the PM2 Configuration

The repo includes an `ecosystem.config.js` but it may need to be updated for standalone mode. Create the production version:

```

cat > /var/www/naro-fashion/ecosystem.config.js << 'PMEOF'
module.exports = {
  apps: [
    {
      name: 'naro-api',
      cwd: '/var/www/naro-fashion/apps/api',
      script: 'dist/main.js',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 4000,
      },
    },
    {
      name: 'naro-storefront',
      cwd: '/var/www/naro-fashion/apps/storefront/.next/standalone/apps/storefront',
      script: 'server.js',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 3000,
        HOSTNAME: '0.0.0.0',
      },
    },
    {
      name: 'naro-admin',
      cwd: '/var/www/naro-fashion/apps/admin/.next/standalone/apps/admin',
      script: 'server.js',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 3001,
        HOSTNAME: '0.0.0.0',
      },
    },
  ],
};
PMEOF

```

Key details:

Setting	Explanation
cwd for storefront/admin	Points to the standalone build directory, NOT the app root
script: 'server.js'	The standalone Next.js server (not node_modules/.bin/next)
HOSTNAME: '0.0.0.0'	Critical for standalone mode — without this, Next.js only listens on 127.0.0.1 and Nginx cannot proxy to it on some configurations
script: 'dist/main.js'	NestJS compiles to JavaScript in dist/ — PM2 runs the compiled output

5.2 Copy Static Files

This must be done after every build, before starting PM2:

```
cd /var/www/naro-fashion

# Storefront static assets
cp -r apps/storefront/.next/static apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public

# Admin static assets
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public
```

5.3 Start All Applications

```
cd /var/www/naro-fashion

# Start all 3 apps
pm2 start ecosystem.config.js

# Save the process list (so PM2 remembers what to run on reboot)
pm2 save

# Enable PM2 to start on boot
pm2 startup
# This prints a command like:
# sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u root --hp /root
# Copy and run that exact command!
```

5.4 Verify All Apps Are Running

```
pm2 list
```

You should see:

id	name	mode	status	cpu	mem	restarts
0	naro-api	fork	online	0.5%	120 MB	0
1	naro-storefront	fork	online	0.3%	80 MB	0
2	naro-admin	fork	online	0.2%	70 MB	0

All 3 must show `online` status with 0 restarts. If any shows `errored` or has a high restart count, check logs:

```
pm2 logs naro-api --lines 50      # API logs
pm2 logs naro-storefront --lines 50 # Storefront logs
pm2 logs naro-admin --lines 50     # Admin logs
```

5.5 Common PM2 Commands

```
pm2 list      # Show status of all apps
pm2 logs      # Stream all logs (Ctrl+C to stop)
```

```
pm2 logs naro-api          # Stream API logs only
pm2 restart all            # Restart all apps
pm2 restart naro-api       # Restart one app
pm2 stop all               # Stop all apps
pm2 delete all             # Remove all apps from PM2
pm2 monit                  # Live CPU/memory monitor
pm2 info naro-api          # Detailed info for one app
pm2 save                   # Save current process list
```

Part 6: Nginx Configuration

Nginx acts as a reverse proxy — it receives all HTTP/HTTPS traffic and routes it to the correct application based on the domain name.

6.1 Create the Nginx Configuration

```
cat > /etc/nginx/sites-available/narofashion << 'NGINX'
# =====
# Storefront — narofashion.co.tz
# =====
server {
    listen 80;
    server_name narofashion.co.tz www.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# =====
# Admin Dashboard — admin.narofashion.co.tz
# =====
server {
    listen 80;
    server_name admin.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```



```
# =====
# API – api.narofashion.co.tz
# =====
server {
    listen 80;
    server_name api.narofashion.co.tz;

    # Allow large file uploads (product images, 3D models up to 25MB)
    client_max_body_size 30M;

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Cache static uploads (product images, payment icons) for 7 days
    location /uploads/ {
        proxy_pass http://127.0.0.1:4000/uploads/;
        add_header Cache-Control "public, max-age=604800";
    }
}
NGINX
```

Replace `narofashion.co.tz` with your actual domain throughout the file if different.

Key settings explained:

Setting	Why
<code>proxy_http_version 1.1</code>	Enables WebSocket support and keep-alive connections
<code>proxy_set_header Host \$host</code>	Passes the original domain to the app (needed for tenant resolution)
<code>proxy_set_header X-Real-IP</code>	Passes the real client IP (not Nginx's IP) to the app
<code>proxy_set_header X-Forwarded-Proto</code>	Tells the app whether the request came via HTTP or HTTPS
<code>client_max_body_size 30M</code>	API accepts file uploads up to 25MB (3D models). 30M gives buffer for multipart overhead
<code>/uploads/ With Cache-Control</code>	Product images and payment icons are cached by the browser for 7 days

6.2 Enable the Site

```
# Create symlink to enable the site
ln -sf /etc/nginx/sites-available/narofashion /etc/nginx/sites-enabled/

# Remove the default Nginx welcome page
```

```
rm -f /etc/nginx/sites-enabled/default

# Test the configuration for syntax errors
nginx -t
# Expected: nginx: configuration file /etc/nginx/nginx.conf test is successful

# Reload Nginx to apply
systemctl reload nginx
```

6.3 Verify Nginx Is Working

At this point (before SSL), you can test with the server IP:

```
# From the server itself:
curl -s http://127.0.0.1:3000 | head -5 # Storefront
curl -s http://127.0.0.1:3001 | head -5 # Admin
curl -s http://127.0.0.1:4000/api/v1 | head -5 # API
```

If any of these return nothing or errors, check `pm2 list` and `pm2 logs`.

Part 7: DNS Setup

DNS tells the internet where to find your domain. You need 4 A records pointing to your VPS IP.

7.1 DNS Records Needed

Type	Name/Host	Value/Points to	TTL
A	@ (root domain)	YOUR_VPS_IP (e.g., 80.240.30.107)	3600
A	www	YOUR_VPS_IP	3600
A	admin	YOUR_VPS_IP	3600
A	api	YOUR_VPS_IP	3600

7.2 Option A: Use Vultr DNS (Recommended)

If your domain registrar does not have a DNS zone editor (common with Tanzanian registrars like Habari Node), point your nameservers to Vultr and manage DNS there.

Step 1: Add domain to Vultr DNS

1. Log in to <https://my.vultr.com>
2. Go to **Products > Network > DNS**
3. Click **Add Domain**
4. Enter your domain: `narofashion.co.tz`
5. Select your VPS as the default IP

Step 2: Add A records

In Vultr DNS for your domain, add:

- @ → YOUR_VPS_IP (A record)
- www → YOUR_VPS_IP (A record)
- admin → YOUR_VPS_IP (A record)
- api → YOUR_VPS_IP (A record)

Optionally add a wildcard for future tenant domains:

- * → narofashion.co.tz (CNAME record)

Step 3: Point nameservers at your registrar

Log in to your domain registrar (e.g., Habari Node at `hosting.habari.co.tz`) and change the nameservers to:

- `ns1.vultr.com`
- `ns2.vultr.com`

Habari Node specific: If you cannot find the nameserver settings, contact their support:

- Portal: <https://hosting.habari.co.tz/> (open a support ticket)
- Email: software@habari.co.tz
- Phone: +255 659 074 444

Tell them: "I need to change nameservers for narofashion.co.tz to ns1.vultr.com and ns2.vultr.com."

7.3 Option B: Use Cloudflare (Free Alternative)

Cloudflare provides free DNS with additional features like DDoS protection and CDN.

1. Go to <https://dash.cloudflare.com> and create a free account
2. Click **Add Site** and enter your domain
3. Select the **Free** plan
4. Add the 4 A records listed in Section 7.1
5. Cloudflare will give you 2 nameservers (e.g., `anna.ns.cloudflare.com` , `bob.ns.cloudflare.com`)
6. Go to your registrar and change nameservers to the Cloudflare nameservers

Note: If using Cloudflare, make sure the proxy (orange cloud) is set to "DNS only" (grey cloud) for the API subdomain, otherwise WebSocket connections and file uploads may have issues.

7.4 Option C: Direct DNS at Registrar

If your registrar provides a DNS zone editor:

1. Log in to your registrar's control panel
2. Find the DNS management / Zone Editor
3. Add the 4 A records from Section 7.1
4. Delete any existing A records that point elsewhere

7.5 Verify DNS Propagation

After making DNS changes, wait 5-30 minutes for A record changes, or up to 24-48 hours for nameserver changes.

```
# Test from any terminal (or use the server):
dig narofashion.co.tz +short
# Expected: 80.240.30.107

dig admin.narofashion.co.tz +short
# Expected: 80.240.30.107

dig api.narofashion.co.tz +short
# Expected: 80.240.30.107
```

If `dig` is not available, use `nslookup` :

```
nslookup narofashion.co.tz
```

Or check online at <https://dnschecker.org> to see global propagation.

DO NOT proceed to SSL setup (Part 8) until DNS is fully propagated. Certbot will fail if the domain does not resolve to your server IP.

Part 8: SSL Certificates

Let's Encrypt provides free SSL certificates. Certbot automates the entire process.

8.1 Install SSL Certificates

```
certbot --nginx \
  -d narofashion.co.tz \
  -d www.narofashion.co.tz \
  -d admin.narofashion.co.tz \
  -d api.narofashion.co.tz \
  --email your-email@example.com \
  --agree-tos \
  --non-interactive
```

Replace:

- Domain names with your actual domain
- Email with your actual email (Let's Encrypt sends expiry reminders here)

On success, Certbot will:

1. Obtain certificates for all 4 domains
2. Automatically modify the Nginx config to add SSL listeners (port 443)
3. Add HTTP-to-HTTPS redirects (port 80 redirects to 443)

8.2 Verify Auto-Renewal

```
certbot renew --dry-run
```

This simulates a renewal. If it says "Congratulations, all simulated renewals succeeded", auto-renewal is working. Certbot installs a systemd timer that runs twice daily and renews certificates before they expire (every 90 days).

8.3 Troubleshooting SSL

"Challenge failed for domain..."

This means Certbot could not verify you own the domain. Causes:

- DNS not propagated yet (most common). Wait and retry.
- Wrong A record (points to wrong IP). Verify with `dig domain +short`.
- Firewall blocking port 80. Check: `ufw status` should show `Nginx Full` allowed.

Certificates for fewer domains than requested:

If one domain fails, Certbot may succeed for the others. Run Certbot again with just the failed domain:

```
certbot --nginx -d admin.narofashion.co.tz --email your@email.com --agree-tos
```

View installed certificates:

Part 9: Email Setup (Brevo)

Brevo (formerly Sendinblue) provides free transactional email (300 emails/day).

9.1 Create Brevo Account

1. Go to <https://www.brevo.com> and sign up (Google OAuth works)
2. Choose the **Free plan** — scroll down past the paid plans; the free option is at the bottom
3. Complete the setup wizard

9.2 Generate SMTP Key

1. In the Brevo dashboard, click the **gear icon** (Settings) in the bottom-left
2. Navigate to **SMTP & API > SMTP**
3. Click **"Generate a new SMTP key"**
4. Name it `naro-fashion-prod`, select **Standard** variant
5. Click **Generate** and **copy the key immediately** (it will not be shown again)

You will have:

```
SMTP Server: smtp-relay.brevo.com
Port:       587
Login:      your-account-id@smtp-brevo.com (shown on the SMTP page)
Password:   xsmtpsib-<long-key>
```

9.3 Add Sender in Brevo

1. Go to **Settings > Senders, Domains & Dedicated IPs > Senders**
2. Click **Add a sender**
3. Enter:
 - **From Name:** `Naro Fashion`
 - **From Email:** `noreply@yourdomain.com` (e.g., `noreply@narofashion.co.tz`)
4. Click **Add**
5. Choose **"Authenticate the domain yourself" > Continue**

9.4 Authenticate Domain (DNS Records)

Brevo will show you DNS records to add. Go to your DNS provider (Vultr DNS, Cloudflare, or your registrar) and add:

Record 1 — Brevo verification code:

Type	Name	Value	TTL
TXT	@ (or blank)	brevo-code:<your-verification-code>	3600

Record 2 — DKIM signature 1:

Type	Name	Value	TTL
CNAME	brevo1._domainkey	b1.yourdomain-tld.dkim.brevo.com	3600

Record 3 — DKIM signature 2:

Type	Name	Value	TTL
------	------	-------	-----

CNAME	brevo2._domainkey	b2.yourdomain-tld.dkim.brevo.com	3600
-------	-------------------	----------------------------------	------

Note: For `.co.tz` domains, the DKIM value uses hyphens: e.g., `b1.narofashion-co-tz.dkim.brevo.com`

Record 4 — DMARC policy:

Type	Name	Value	TTL
TXT	_dmarc	v=DMARC1; p=none; rua=mailto:rua@dmarc.brevo.com	3600

9.5 Verify Domain in Brevo

1. Go back to Brevo and click **Authenticate / Check Configuration**
2. Wait a few minutes for DNS to propagate
3. Once verified, your domain shows a green checkmark
4. Emails from `noreply@yourdomain.com` will now pass SPF, DKIM, and DMARC checks (delivered to inbox, not spam)

9.6 Update Environment Variables on VPS

```
ssh root@YOUR_VPS_IP

cd /var/www/naro-fashion
nano .env

# Update these lines:
# SMTP_HOST="smtp-relay.brevo.com"
# SMTP_PORT="587"
# SMTP_USER="your-account-id@smtp-brevo.com"
# SMTP_PASS="xsmtpsib-your-key-here"
# SMTP_FROM="noreply@yourdomain.com"
```

Save the file, then copy to API and restart:

```
cp .env apps/api/.env
pm2 restart naro-api
```

9.7 What Emails the Platform Sends

Email Type	Trigger	Template
Order confirmation	Customer places an order	order-confirmation.hbs
Password reset	User clicks "Forgot Password"	password-reset.hbs
Rental pickup reminder	Daily cron (8:00 AM) for upcoming pickups	rental-reminder.hbs
Overdue rental alert	Daily cron (9:00 AM) for overdue returns	overdue-rental.hbs
Admin prep reminder	Item needs preparation before pickup	admin-prep-reminder.hbs
ID verification status	Admin approves/rejects National ID	id-verification.hbs
Newsletter campaign	Admin sends via newsletter module	newsletter.hbs
Contact acknowledgement	Customer submits contact form	contact-acknowledgement.hbs
Contact reply	Admin replies to contact submission	contact-reply.hbs

Part 10: Tenant Configuration

The storefront resolves which tenant to show based on the domain. You must set the tenant's domain in the database.

10.1 Set Tenant Domain

```
# Connect to the database and update the tenant domain
sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" = 'yourdomain.com' WHERE
\"slug\" = 'naro-fashion';"
```

Replace `yourdomain.com` with your actual domain (e.g., `narofashion.co.tz`). Do NOT include `https://` or `www.`.

```
# Restart the storefront so middleware picks up the domain
pm2 restart naro-storefront
```

10.2 Verify Tenant Resolution

```
# Test that the storefront can resolve the tenant
curl -s https://yourdomain.com | head -20
# Should return HTML (not a "Store not found" error)
```

10.3 Why Not Use a Node Script?

You might think of running a Prisma script to update the tenant. **Do not use `node -e` scripts with Prisma on the server from arbitrary directories** (like `/tmp`). pnpm's `node_modules` resolution is directory-dependent, and `@prisma/client` will not be found outside the project directory. Using `psql` directly is the most reliable method.

Part 11: CI/CD Pipeline

The project uses GitHub Actions to automatically deploy when you push to the `prod` branch.

11.1 How It Works

```
Developer pushes to prod branch
|
▼
GitHub Actions triggers deploy-prod.yml
|
▼
SSH into VPS and run deploy.sh
|
▼
deploy.sh: git pull → pnpm install → prisma generate →
           prisma db push → pnpm build → copy static → pm2 restart
```

11.2 Set Up SSH Key for GitHub Actions

On the VPS:

```
# Generate a deploy key (no passphrase)
ssh-keygen -t ed25519 -f ~/.ssh/deploy_key -N '' -C 'github-actions-deploy'
```

```
# Allow this key to SSH into the server
cat ~/.ssh/deploy_key.pub >> ~/.ssh/authorized_keys
```

11.3 Configure GitHub Secrets

Go to your GitHub repository > **Settings** > **Secrets and variables** > **Actions** > **New repository secret** and add:

Secret Name	Value
VPS_HOST	Your VPS IP (e.g., 80.240.30.107)
VPS_USER	root
VPS_SSH_KEY	The private key content from the server: run <code>cat ~/.ssh/deploy_key</code> on the VPS and paste the entire output including the -----BEGIN/END----- lines

Or use the GitHub CLI (if authenticated):

```
# Run these from your local machine
echo "80.240.30.107" | gh secret set VPS_HOST
echo "root" | gh secret set VPS_USER

# Get the private key from the server and set it:
ssh root@80.240.30.107 "cat ~/.ssh/deploy_key" | gh secret set VPS_SSH_KEY

# Verify all secrets are set:
gh secret list
```

11.4 The GitHub Actions Workflow

The workflow file is at `.github/workflows/deploy-prod.yml` :

```
name: Deploy to Production

on:
  push:
    branches: [prod]

jobs:
  deploy:
    name: Deploy to VPS
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }
          username: ${ secrets.VPS_USER }
          key: ${ secrets.VPS_SSH_KEY }
          port: 22
          script_stop: true
          script: |
            cd /var/www/naro-fashion
```



```
chmod +x deploy.sh
bash deploy.sh
```

This uses the `appleboy/ssh-action` to SSH into the VPS and run the deploy script. `script_stop: true` means it stops on any error (respects `set -e` in the deploy script).

11.5 The Deploy Script

The deploy script at `/var/www/naro-fashion/deploy.sh` handles the full deployment:

```
#!/bin/bash
set -e

echo "Deploying Naro Fashion..."
cd /var/www/naro-fashion

# Backup .env files (git reset --hard deletes untracked files)
cp apps/storefront/.env.local /tmp/storefront.env.local 2>/dev/null || true
cp apps/admin/.env.local /tmp/admin.env.local 2>/dev/null || true
cp .env /tmp/naro.env 2>/dev/null || true
cp apps/api/.env /tmp/api.env 2>/dev/null || true

# Pull latest code (hard reset to match remote exactly)
git fetch origin prod
git reset --hard origin/prod

# Restore .env files
cp /tmp/storefront.env.local apps/storefront/.env.local 2>/dev/null || true
cp /tmp/admin.env.local apps/admin/.env.local 2>/dev/null || true
cp /tmp/naro.env .env 2>/dev/null || true
cp /tmp/api.env apps/api/.env 2>/dev/null || true
cp .env packages/database/.env 2>/dev/null || true

# Install dependencies
pnpm install

# Generate Prisma client and push schema changes
cd packages/database
npx prisma generate
npx prisma db push --accept-data-loss
cd ../../

# Build all 3 apps
pnpm build

# Copy static files for standalone Next.js
cp -r apps/storefront/.next/static apps/storefront/.next/standalone/apps/storefront/.next/static
2>/dev/null || true
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public 2>/dev/null || true
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static 2>/dev/null || true
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public 2>/dev/null || true

# Restart all PM2 processes
pm2 restart ecosystem.config.js
pm2 save
```

```
echo "Deployment complete! $(date)"
```

Why backup and restore .env files? The `.env` and `.env.local` files are NOT in git (they are in `.gitignore`). The `git reset --hard` command removes all untracked files. Without the backup/restore, every deploy would delete your environment variables.

11.6 How to Deploy

Workflow: develop on `master` (or `dev`), deploy from `prod`

```
# 1. Work on master/dev branch normally
git add .
git commit -m "your changes"
git push origin master

# 2. When ready to deploy to production:
git checkout prod
git merge master --no-edit
git push origin prod    # This triggers the GitHub Actions workflow

# 3. Switch back to your working branch
git checkout master
```

Monitor the deployment:

```
# Check the latest workflow run
gh run list --limit 1

# Watch the logs if it failed
gh run view <run-id> --log
```

Or go to GitHub > your repo > **Actions** tab to see the workflow status.

11.7 Create the prod Branch (First Time Only)

```
# On your local machine, from the master branch:
git checkout master
git checkout -b prod
git push origin prod

# Switch back to your working branch
git checkout master
```

Part 12: Firewall and Security

12.1 Configure UFW Firewall

```
# Allow SSH (port 22)
ufw allow OpenSSH

# Allow HTTP and HTTPS (ports 80 and 443)
ufw allow 'Nginx Full'
```

```
# Enable the firewall
ufw enable
# Type "y" when prompted

# Verify
ufw status
```

Expected output:

```
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
Nginx Full ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
Nginx Full (v6) ALLOW Anywhere (v6)
```

IMPORTANT: Only 3 ports are open: 22 (SSH), 80 (HTTP), 443 (HTTPS). PostgreSQL (5432) is NOT exposed to the internet — it only listens on localhost.

12.2 Secure SSH

```
# Edit SSH configuration
nano /etc/ssh/sshd_config
```

Find and change these lines:

```
PasswordAuthentication no
PermitRootLogin prohibit-password
```

This disables password login (SSH key only) and prevents root login with a password. Save and restart:

```
systemctl restart sshd
```

WARNING: Make sure your SSH key works *BEFORE* disabling password authentication. Test by opening a new Git Bash window and connecting: `ssh root@YOUR_IP`. If it works without a password prompt, you are safe to disable password auth.

12.3 Database Backups

```
# Create backup directory
mkdir -p /var/backups/naro

# Add daily backup cron job (runs at 2:00 AM)
(crontab -l 2>/dev/null; echo '0 2 * * * pg_dump -U naro_admin naro_fashion | gzip >
/var/backups/naro/naro_fashion_$(date +%F).sql.gz') | crontab -

# Add cleanup cron (delete backups older than 30 days, runs at 3:00 AM)
(crontab -l 2>/dev/null; echo '0 3 * * * find /var/backups/naro -name "naro_fashion_*.sql.gz" -mtime +30
-delete') | crontab -
```

```
# Verify cron jobs were added
crontab -l
```

Manual backup:

```
pg_dump -U naro_admin naro_fashion | gzip > /var/backups/naro/naro_fashion_$(date +%F).sql.gz
```

Restore from backup:

```
gunzip < /var/backups/naro/naro_fashion_2026-04-11.sql.gz | sudo -u postgres psql naro_fashion
```

12.4 PM2 Log Rotation

```
pm2 install pm2-logrotate

# Configure: max 10MB per log file, keep 7 rotated files
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
```

12.5 Security Features Already Configured

The application has these security measures built-in:

Feature	Details
Helmet	HTTP security headers (CSP, XSS protection, clickjacking prevention) on the API
Rate Limiting	100 requests per 60 seconds per IP (NestJS Throttler)
Account Lockout	5 failed login attempts result in a 30-minute lock. Tracked in <code>LoginAttempt</code> table
Password Hashing	bcryptjs with salt round 12
CORS	Only storefront and admin URLs are whitelisted
File Upload Limits	5MB for images, 25MB for 3D models
Row-Level Isolation	All data scoped by <code>tenantId</code> — tenants cannot see each other's data
Three-Tier Auth	PlatformAdmin / AdminUser / User — each with different access levels

12.6 Recommended Future Improvements

- **fail2ban:** Protect SSH from brute-force attacks
- **Cloudflare:** Add as CDN/DDoS protection layer in front of Nginx
- **PostgreSQL SSL:** Enable SSL for database connections
- **Offsite Backups:** Copy backups to S3-compatible storage (Vultr Object Storage is \$5/mo for 250GB)
- **Monitoring:** Set up uptime monitoring (e.g., UptimeRobot free tier — 50 monitors)

Part 13: Post-Deployment Verification

Run through this checklist after every deployment.

13.1 Service Health Checks

```
# Check all PM2 processes are running
pm2 list
# All 3 should show "online" with 0 or low restart count

# Test API health
curl -s https://api.yourdomain.com/api/v1 | head -20

# Test storefront
curl -s -o /dev/null -w "%{http_code}" https://yourdomain.com
# Expected: 200

# Test admin
curl -s -o /dev/null -w "%{http_code}" https://admin.yourdomain.com
# Expected: 200

# Test SSL
curl -vI https://yourdomain.com 2>&1 | grep "SSL certificate"
# Should show certificate details
```

13.2 Full Verification Checklist

#	Task	How to Verify	Expected Result
1	Storefront loads	Visit https://yourdomain.com	Homepage renders with products
2	Admin loads	Visit https://admin.yourdomain.com	Login page renders
3	API responds	Visit https://api.yourdomain.com/api/v1	JSON response
4	SSL working	Check padlock icon in browser	Green padlock on all 3 domains
5	Admin login	Email: admin@narofashion.co.tz , Password: admin123	Dashboard loads
6	Platform login	URL: https://admin.yourdomain.com/platform-login , Email: platform@naro.co.tz , Password: Admin123	Platform dashboard loads
7	CHANGE PASSWORDS	Change both admin and platform admin passwords IMMEDIATELY	Confirm new password works
8	Products visible	Browse storefront shop page	Products display with images
9	Images load	Check product images and logos	All images render (no broken images)
10	Mobile works	Open site on phone or use browser responsive mode	Mobile layout correct
11	Backups configured	<code>crontab -l</code> shows backup job	Backup cron entries visible
12	Swap enabled	Run <code>free -h</code>	Swap line shows ~2.0G
13	PM2 auto-start	Run <code>sudo reboot</code> , wait 30 seconds, then verify apps come back	All 3 apps online after reboot

14	Firewall active	ufw status	Shows SSH + Nginx Full allowed
----	-----------------	------------	--------------------------------

13.3 Test Authentication

```
# Test admin login via API
curl -s -X POST https://api.yourdomain.com/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@narofashion.co.tz","password":"admin123"}' | head -100

# Expected: JSON with access_token and refresh_token

# Test platform admin login
curl -s -X POST https://api.yourdomain.com/api/v1/auth/platform-login \
  -H "Content-Type: application/json" \
  -d '{"email":"platform@naro.co.tz","password":"Admin123"}' | head -100

# Expected: JSON with access_token (isPlatformAdmin: true in JWT)
```

Part 14: Known Issues and Solutions

These are real issues encountered during deployment and ongoing operation, with proven solutions.

14.1 TenantContext for @Public() Endpoints

Problem: Endpoints decorated with `@Public()` skip `JwtAuthGuard`, so `req.user` is never populated. But these endpoints still need the `tenantId` for data scoping (e.g., `GET /products`, `GET /cms/pages`).

Solution (already implemented): Both `TenantContext` and `ModuleGuard` have a JWT fallback. When `req.user` is null, they decode the JWT from the `Authorization` header directly (Base64 payload decode, no cryptographic verification needed since it is only used for tenant scoping, not authentication). If CMS pages or product listings return 500 errors, this fallback may have been broken.

14.2 ModuleGuard DI Issues

Problem: `ModuleGuard` originally tried to inject `JwtService` for token decoding. This caused dependency injection failures in non-auth modules because `JwtService` is only provided by `AuthModule`.

Solution (already implemented): `ModuleGuard` uses manual Base64 decoding of the JWT payload (`Buffer.from(payload, 'base64url').toString()`) instead of `JwtService`. No external dependency needed.

14.3 API Crash-Looping (Restart Count > 1000)

Symptom: `pm2 list` shows `naro-api` with a restart count climbing into hundreds or thousands.

Diagnosis:

```
# Check the error
pm2 logs naro-api --lines 50

# Or run the API directly to see the full error:
cd /var/www/naro-fashion/apps/api
node dist/main.js
```

Common causes:

Error	Fix
Cannot find module 'multer'	<code>pnpm add multer @types/multer --filter api</code>
Cannot find module '@prisma/client'	<code>cd packages/database && npx prisma generate</code>
ECONNREFUSED 127.0.0.1:5432	PostgreSQL not running: <code>systemctl start postgresql</code>
DI injection error (NestJS)	Missing provider in a module — check the error message for the class name

After fixing, restart:

```
pm2 restart naro-api
pm2 reset naro-api # Resets the restart counter to 0
```

14.4 502 Bad Gateway After Restart

Symptom: After `pm2 restart all`, visiting any URL shows "502 Bad Gateway" from Nginx.

Cause: Applications take 10-15 seconds to start up. The API is the slowest because it initializes Prisma connections, seeds permissions, and starts cron jobs.

Solution: Wait 15 seconds and try again. If it persists after 30 seconds, check `pm2 list` — one or more apps may have crashed (status = `errored` or high restart count).

14.5 Storefront Shows "Store not found"

Symptom: Storefront loads but shows a "Store not found" or blank error page.

Causes:

1. **Tenant domain not set** — The `Tenant` table has no `domain` value, or it does not match the URL you are visiting. Fix with Part 10.
2. **DNS not pointing to server** — The domain resolves to a different IP. Check with `dig yourdomain.com +short`.
3. **API not running** — The storefront middleware calls the API to resolve the tenant. If the API is down, tenant resolution fails. Check `pm2 list`.

Fix:

```
# Check the tenant domain in the database
sudo -u postgres psql -d naro_fashion -c 'SELECT id, name, slug, domain, status FROM "Tenant";'

# Set/fix the domain
sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" = 'yourdomain.com' WHERE \"slug\" = 'naro-fashion';"

# Restart storefront
pm2 restart naro-storefront
```

14.6 "Tenant context is required but not available"

Symptom: API returns 500 error with message "Tenant context is required but not available".

Cause: An admin user is calling an endpoint that requires `tenantId`, but the request does not have the `X-Tenant-Id` header and the JWT does not contain a `tenantId`. This can happen with:

- Public endpoints called without authentication
- Platform admin calling tenant-scoped endpoints

Solution: This is already handled by the JWT decode fallback in `TenantContext` . If it still occurs, ensure the admin user has a `tenantId` in their JWT (check by decoding the token at <https://jwt.io>).

14.7 Static Assets Not Loading (Blank White Page)

Symptom: The page loads but is completely blank/white. Browser console shows 404 errors for `.js` and `.css` files.

Cause: Static files were not copied to the standalone directory after the build.

Fix:

```
cd /var/www/naro-fashion

# Copy static files
cp -r apps/storefront/.next/static apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public

# Restart
pm2 restart naro-storefront naro-admin
```

Part 15: Updating the Application

15.1 Via CI/CD (Recommended)

The easiest way to deploy updates:

```
# On your local machine:
git checkout master
# ... make changes, commit ...
git push origin master

# When ready to deploy:
git checkout prod
git merge master --no-edit
git push origin prod    # Triggers automatic deployment

# Monitor:
gh run list --limit 1

# Switch back:
git checkout master
```

15.2 Manual Update (SSH into server)

```
ssh root@YOUR_VPS_IP
cd /var/www/naro-fashion

# Pull latest code
git pull origin master

# Install any new dependencies
pnpm install
```



```
# Update database schema
cd packages/database
npx prisma generate
npx prisma db push --accept-data-loss
cd ../../

# Build
pnpm build

# Copy static files (CRITICAL – do not skip)
cp -r apps/storefront/.next/static apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public

# Restart all apps
pm2 restart all
```

15.3 Using the Deploy Script

```
ssh root@YOUR_VPS_IP
cd /var/www/naro-fashion
chmod +x deploy.sh
./deploy.sh
```

The deploy script handles everything: env backup, git pull, install, prisma, build, static copy, pm2 restart.

15.4 Database-Only Changes

If you only changed the Prisma schema (no code changes):

```
cd /var/www/naro-fashion/packages/database
npx prisma generate
npx prisma db push --accept-data-loss
cd ../../
pm2 restart naro-api
```

15.5 API-Only Restart

If you changed environment variables (no code changes):

```
# Edit the .env file
nano /var/www/naro-fashion/.env
cp /var/www/naro-fashion/.env /var/www/naro-fashion/apps/api/.env
pm2 restart naro-api
```

Part 16: Cost Summary

16.1 Monthly Costs

Service	Provider	Monthly Cost	Notes
VPS	Vultr (vhf-1c-2gb)	\$12.00	1 vCPU, 2GB RAM, 64GB NVMe, Frankfurt

Domain	Habari Node (.co.tz)	~\$1.76	TZS 21,186/year (~\$8.25/year)
SSL	Let's Encrypt	Free	Auto-renews every 90 days
Email	Brevo (free tier)	Free	300 emails/day
DNS	Vultr DNS	Free	Included with VPS
CI/CD	GitHub Actions	Free	2,000 minutes/month on free plan
TOTAL		~\$13.76/mo	~\$152.25/year

16.2 Scaling Costs

If you outgrow the current server:

Plan	Specs	Cost	When to upgrade
Current (vhf-1c-2gb)	1 vCPU, 2GB RAM	\$12/mo	Up to ~1,000 concurrent users
vhf-2c-4gb	2 vCPU, 4GB RAM	\$24/mo	Heavy traffic, many tenants
vhf-4c-8gb	4 vCPU, 8GB RAM	\$48/mo	10+ tenants, analytics workloads

16.3 Adding a New Tenant Client

When selling the platform to a new client:

1. **Login** as platform admin at `https://admin.yourdomain.com/platform-login`
2. Go to **Tenants** > **New Tenant**
3. Fill in: company name, slug, admin email/password, subscription plan
4. The new tenant gets their own isolated storefront, admin, and data

To connect a **custom domain** for the new tenant:

```
# 1. Add their domain as an A record in DNS pointing to your VPS IP
# 2. Get SSL for the new domain:
certbot --nginx -d newclient.co.tz

# 3. Set the domain in the database:
sudo -u postgres psql -d naro_fashion \
  -c "UPDATE \"Tenant\" SET \"domain\" = 'newclient.co.tz' WHERE \"slug\" = 'new-client-slug';"

# 4. Restart storefront:
pm2 restart naro-storefront
```

Appendix A: All Environment Variables Reference

Root `.env` (also copied to `packages/database/.env` and `apps/api/.env`)

Variable	Required	Example	Description
NODE_ENV	Yes	production	Runtime environment. Set to production ON server

DATABASE_URL	Yes	postgresql://naro_admin:Pass@localhost:5432/naro_fashion?schema=public	PostgreSQL connection string
JWT_SECRET	Yes	aB3dE5fG7hI9... (64 chars)	Secret key for signing JWT access tokens
JWT_REFRESH_SECRET	Yes	zY9xW7vU5tS3... (64 chars)	Secret key for signing JWT refresh tokens. Must differ from JWT_SECRET
JWT_EXPIRATION	Yes	15m	Access token lifetime (15 minutes)
JWT_REFRESH_EXPIRATION	Yes	7d	Refresh token lifetime (7 days)
STOREFRONT_URL	Yes	https://narofashion.co.tz	Public URL of the storefront (used for CORS, email links)
ADMIN_URL	Yes	https://admin.narofashion.co.tz	Public URL of the admin dashboard (used for CORS)
API_URL	Yes	https://api.narofashion.co.tz	Public URL of the API
NEXT_PUBLIC_API_URL	Yes	https://api.narofashion.co.tz/api/v1	API URL with prefix (used by frontend apps)
SMTP_HOST	No*	smtp-relay.brevo.com	SMTP server hostname. *Required for emails to work
SMTP_PORT	No*	587	SMTP server port (587 for TLS)
SMTP_USER	No*	a58aa0001@smtp-brevo.com	SMTP authentication username
SMTP_PASS	No*	xsmtpsib-xxxx...	SMTP authentication password/key
SMTP_FROM	No*	noreply@narofashion.co.tz	Sender email address for outgoing emails

INSTAGRAM_BUSINESS_ACCOUNT_ID	No	17841418108905851	Instagram Business Account ID for feed sync
FACEBOOK_APP_ID	No	4338851449722487	Facebook App ID for Instagram API access
INSTAGRAM_ACCESS_TOKEN	No	EAAx...	Long-lived Facebook Graph API access token
FACEBOOK_APP_SECRET	No	abc123...	Facebook App Secret for token refresh

Storefront `.env.local` (`apps/storefront/.env.local`)

Variable	Required	Example	Description
NEXT_PUBLIC_API_URL	Yes	https://api.narofashion.co.tz/api/v1	API URL with /api/v1 prefix
NEXT_PUBLIC_TENANT_SLUG	Yes	naro-fashion	Fallback tenant slug (used when domain resolution fails)

Admin `.env.local` (`apps/admin/.env.local`)

Variable	Required	Example	Description
NEXT_PUBLIC_API_URL	Yes	https://api.narofashion.co.tz/api/v1	API URL with /api/v1 prefix

Appendix B: Quick Reference Commands

PM2 (Process Manager)

```

pm2 list           # Show all processes with status
pm2 logs           # Stream all logs (Ctrl+C to stop)
pm2 logs naro-api --lines 50 # Last 50 lines of API logs
pm2 restart all    # Restart all apps
pm2 restart naro-api # Restart just the API
pm2 restart naro-storefront # Restart just the storefront
pm2 restart naro-admin # Restart just the admin
pm2 stop all       # Stop all apps
pm2 delete all     # Remove all from PM2
pm2 start ecosystem.config.js # Start all from config
pm2 save           # Save current process list
pm2 startup        # Generate startup script
pm2 monit          # Live CPU/memory monitor
pm2 info naro-api # Detailed process info
pm2 reset naro-api # Reset restart counter
pm2 flush          # Clear all log files

```

Nginx

```
nginx -t                                # Test config syntax
systemctl reload nginx                  # Reload config (no downtime)
systemctl restart nginx                 # Full restart
systemctl status nginx                  # Check status
cat /etc/nginx/sites-enabled/narofashion # View current config
nano /etc/nginx/sites-available/narofashion # Edit config
```

PostgreSQL

```
sudo -u postgres psql                  # Open psql as superuser
sudo -u postgres psql -d naro_fashion  # Open psql to naro_fashion database
sudo -u postgres psql -d naro_fashion -c 'SELECT * FROM "Tenant";' # Run a query
systemctl status postgresql            # Check status
systemctl restart postgresql           # Restart
systemctl start postgresql             # Start
```

Common SQL queries:

```
-- List all tenants
SELECT id, name, slug, domain, status FROM "Tenant";

-- List all admin users
SELECT id, email, "firstName", "lastName", role, "tenantId" FROM "AdminUser";

-- List platform admins
SELECT id, email, "firstName", "lastName" FROM "PlatformAdmin";

-- Check enabled modules for a tenant
SELECT tm.*, md.name FROM "TenantModule" tm
JOIN "ModuleDefinition" md ON md.code = tm."moduleCode"
WHERE tm."tenantId" = '<tenant-id>' AND tm.enabled = true;

-- Count records in key tables
SELECT
  (SELECT COUNT(*) FROM "Product") as products,
  (SELECT COUNT(*) FROM "Order") as orders,
  (SELECT COUNT(*) FROM "User") as customers,
  (SELECT COUNT(*) FROM "Rental") as rentals;
```

Certbot (SSL)

```
certbot certificates                    # List all certificates + expiry dates
certbot renew --dry-run                 # Test auto-renewal
certbot renew --force-renewal           # Force renew all certificates
certbot --nginx -d newdomain.com        # Add SSL for a new domain
```

UFW (Firewall)

```
ufw status                             # Show current rules
ufw allow OpenSSH                       # Allow SSH
ufw allow 'Nginx Full'                  # Allow HTTP + HTTPS
```

```
ufw enable          # Enable firewall
ufw disable         # Disable firewall (careful!)
```

System

```
free -h            # Check RAM and swap usage
df -h              # Check disk space
htop               # Interactive process monitor
uptime            # Server uptime and load
du -sh /var/www/naro-fashion # Check app directory size
du -sh /var/backups/naro    # Check backup directory size
cat /etc/os-release        # Check OS version
node -v                   # Check Node.js version
pnpm -v                   # Check pnpm version
```

Prisma (Database)

```
cd /var/www/naro-fashion/packages/database

npx prisma generate      # Regenerate Prisma client
npx prisma db push --accept-data-loss # Push schema to database
npx prisma studio        # Open visual database browser (port 5555)
npx prisma migrate deploy # Run pending migrations
```

Git

```
cd /var/www/naro-fashion

git status          # Check current state
git log --oneline -10 # Last 10 commits
git pull origin master # Pull latest from master
git branch -a        # List all branches
```

Infrastructure Reference

Current Production Setup

Component	Details
VPS Provider	Vultr (vultr.com)
Server Plan	vhf-1c-2gb (1 vCPU, 2GB RAM, 64GB NVMe SSD)
Server Location	Frankfurt, Germany
Server IP	80.240.30.107
OS	Ubuntu 24.04 LTS
Domain Registrar	Habari Node (habarinode.co.tz)
DNS Provider	Vultr DNS (ns1.vultr.com, ns2.vultr.com)
SSL	Let's Encrypt (auto-renews every 90 days)

Email	Brevo SMTP (smtp-relay.brevo.com, 300/day free)
Node.js	v22.x LTS
Package Manager	pnpm v10.x
Process Manager	PM2
Web Server	Nginx
Database	PostgreSQL 16
App Directory	/var/www/naro-fashion

Default Credentials (CHANGE IMMEDIATELY AFTER FIRST LOGIN)

Login Type	Email	Password	URL
Tenant Admin	admin@narofashion.co.tz	admin123	https://admin.yourdomain.com/login
Platform Admin	platform@naro.co.tz	Admin123	https://admin.yourdomain.com/platform-login

DNS Records (Vultr DNS)

Type	Name	Value	TTL
A	@	80.240.30.107	300
A	www	80.240.30.107	3600
A	admin	80.240.30.107	3600
A	api	80.240.30.107	3600
CNAME	*	narofashion.co.tz	300
NS		ns1.vultr.com	300
NS		ns2.vultr.com	300